



Politechnika Wrocławska

RAPORT

PROJEKT 2

PROJEKTOWANIE I ANALIZA ALGORYTMÓW

30 KWIETNIA 2026

Danylo Mikhnytskyi
Indeks: 284125

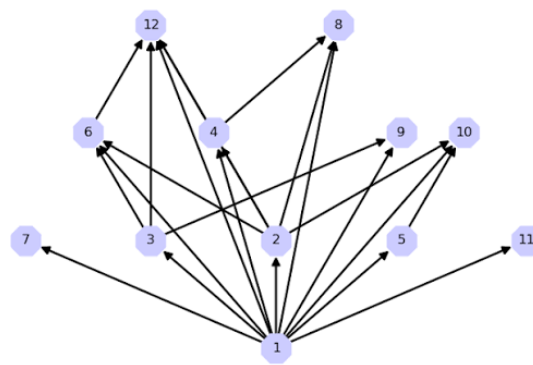
Spis treści

1	Wprowadzenie	2
1.1	Omówienie ćwiczenia	2
1.2	Wybrany algorytm (Dijkstra & Bellman-Ford)	2
1.2.1	Zasada działania	2
1.2.2	Własności i Złożoność obliczeniowa	2
1.2.3	Zastosowania	3
2	Implementacje grafu	3
2.1	Lista sąsiedztwa	4
2.2	Macierz sąsiedztwa	5
3	Testy efektywności	5
3.1	Wyniki pomiarów w formie tabelarycznej	5
3.2	Wykresy typu 1: Względem reprezentacji	6
3.3	Wykresy typu 2: Względem gęstości grafu	8
4	Wnioski	10
5	Bibliografia	11

1 Wprowadzenie

1.1 Omówienie ćwiczenia

Celem ćwiczenia było zaprojektowanie, zaimplementowanie i przetestowanie aplikacji konsolowej w języku C++ rozwiązującej problem najkrótszej ścieżki (Shortest Path Problem) w grafach skierowanych. Projekt wymagał zastosowania metodologii programowania obiektowego do reprezentacji grafu (za pomocą macierzy sąsiedztwa i listy sąsiedztwa) oraz wykorzystania dwóch algorytmów: algorytmu Dijkstry (wykorzystującego kopiec do optymalizacji) oraz algorytmu Bellmana-Forda. Dodatkowo należało stworzyć zautomatyzowany system do testowania wydajności, sprawdzający czas wykonania algorytmów na wygenerowanych losowo grafach o zadanej liczbie wierzchołków oraz gęstości.



Rysunek 1: Przykładowy graf skierowany ważony.

1.2 Wybrany algorytm (Dijkstra & Bellman-Ford)

1.2.1 Zasada działania

Algorytm Dijkstry: Służy do znajdowania najkrótszych ścieżek z pojedynczego wierzchołka źródłowego do wszystkich pozostałych wierzchołków w grafie o nieujemnych wagach krawędzi. Działa w oparciu o strategię zachłanną. Utrzymuje zbiór wierzchołków, dla których minimalna odległość została już znaleziona, oraz kolejkę priorytetową (kopiec typu *min-heap*), w której przechowuje kandydatów do sprawdzenia. W każdym kroku wybiera wierzchołek o najmniejszej znanej odległości, dodaje go do przetworzonych i aktualizuje (*relaksuje*) odległości do jego sąsiadów.

Algorytm Bellmana-Forda: Rozwiązuje ten sam problem, ale jest bardziej uniwersalny – dopuszcza krawędzie o wagach ujemnych. Algorytm relaksuje wszystkie krawędzie w grafie $V - 1$ razy (gdzie V to liczba wierzchołków). Jeśli po tych operacjach jakkolwiek krawędź może zostać poddana relaksacji po raz kolejny, oznacza to, że w grafie występuje cykl o ujemnej wadze. W zaimplementowanym kodzie zoptymalizowano działanie, dodając warunek przerwania algorytmu we wcześniejszej iteracji, jeśli nie nastąpiła żadna zmiana dystansu.

1.2.2 Własności i Złożoność obliczeniowa

Algorytm Dijkstry: Wykorzystana w projekcie implementacja oparta jest na wbudowanej kolejce priorytetowej `std::priority_queue` w języku C++. Zastosowano także

technikę leniwego usuwania wierzchołków z kolejki (ang. *lazy deletion*), polegającą na odrzucaniu nieaktualnych wartości przy ich wyciągnięciu.

- Złożoność czasowa: Dla listy sąsiedztwa i zastosowanego kopca binarnego wynosi optymistycznie $O((V + E) \log V)$ (lub $O(E \log V)$ bo $E \geq V$). Z kolei przy zastosowaniu macierzy sąsiedztwa, uzyskanie wszystkich sąsiadów danego wierzchołka wymaga przejrzania całego wiersza tablicy o wielkości V , co sprawia, że sumaryczna złożoność wzrasta i jest ograniczona z dołu przez $O(V^2 + E \log V)$.
- Złożoność pamięciowa: $O(V)$ dodatkowej pamięci niezbędnej do przechowywania odległości, tablicy poprzedników i elementów znajdujących się w kopcu.

Algorytm Bellmana-Forda:

- Złożoność czasowa: Wynosi $O(V \cdot E)$. Złożoność ta wynika z pętli zewnętrznej wykonującej się co najwyżej $V - 1$ razy i w każdym przejściu sprawdzającej wszystkie E krawędzi.
- Złożoność pamięciowa: $O(V)$ ze względu na tablicę zapisanych odległości `dist` i tablicę pamiętającą strukturę wyznaczonej ścieżki `prev`.

1.2.3 Zastosowania

Ze względu na wysoką efektywność, algorytm Dijkstry jest powszechnie stosowany w wielu dziedzinach, w których wagi mogą być wyłącznie nieujemne. Stosowany jest m.in. w nawigacji satelitarnej, Google Maps, a także w trasowaniu (protokół OSPF).

Algorytm Bellmana-Forda nie jest zazwyczaj głównym wyborem w zagadnieniach bez wag ujemnych ze względu na swoją dużą złożoność pesymistyczną. Stosuje się go w sytuacjach dopuszczających takowe wagi, czyli do trasowania w internecie (np. protokół RIP używający odmiany algorytmu), czy podczas wykrywania arbitrażu rynkowego.

2 Implementacje grafu

Zaprojektowana architektura programu opiera się na zastosowaniu podejścia obiektowego. Graf reprezentowany jest jako abstrakcyjna klasa bazowa `Graph`, która zawiera metody wirtualne implementowane następnie przez konkretne klasy: `AdjList` i `AdjMatrix`. Umożliwia to zunifikowany dostęp dla poszczególnych algorytmów.

Fragment kodu 1: Implementacja abstrakcyjnej klasy `Graph`.

```
1 // Represents a directed weighted edge
2 struct Edge {
3     int from;
4     int to;
5     int weight;
6 };
7
8 // Abstract base class for a directed weighted graph
9 class Graph {
10 protected:
11     int vertexCount;
12     int edgeCount;
13 }
```

```

14 public:
15     explicit Graph(int vertices)
16         : vertexCount(vertices), edgeCount(0) {}
17     virtual ~Graph() = default;
18     virtual void addEdge(int u, int v, int w) = 0;
19     virtual std::vector<std::pair<int, int>> getNeighbors(int v) const =
20         0;
21     virtual int getEdgeWeight(int u, int v) const = 0;
22     virtual std::vector<Edge> getAllEdges() const = 0;
23     // ...
};

```

Badania grafów ułatwia dedykowana funkcja generatora losowego. Po wyliczeniu maksymalnej liczby krawędzi funkcja tworzy pulę wszystkich potencjalnych relacji (pomijając pętle, tzw. self-loops). Następnie z wykorzystaniem optymalnego algorytmu Fishera-Yatesa (wywołanie `std::shuffle`) pula ta jest tasowana i pobierane jest pierwsze E krawędzi, gwarantując dokładnie żadaną gęstość i brak zduplikowanych par wierzchołków.

Fragment kodu 2: Funkcja do generowania losowego grafu skierowanego.

```

1  std::unique_ptr<Graph> generateRandomGraph(int V, double density,
2  RepType rep, bool allowNegative) {
3      long long maxEdges = static_cast<long long>(V) * (V - 1);
4      int E = static_cast<int>(std::round(density * maxEdges));
5
6      std::vector<std::pair<int, int>> allEdges;
7      allEdges.reserve(maxEdges);
8      for (int u = 0; u < V; ++u) {
9          for (int v = 0; v < V; ++v) {
10             if (u != v) allEdges.emplace_back(u, v);
11         }
12     }
13
14     static std::mt19937 rng(std::random_device{}());
15     std::shuffle(allEdges.begin(), allEdges.end(), rng);
16     // ... wstawienie wylosowanych krawedzi do grafu
};

```

2.1 Lista sąsiedztwa

W liście sąsiedztwa poszczególne krawędzie są zapisane wewnątrz wektora, gdzie indeks zewnętrzny jednoznacznie określa wierzchołek źródłowy, a elementem jest odpowiednia para obejmująca wierzchołek docelowy i powiązaną wagę. Klasa dziedziczy publicznie po `Graph` i inicjuje listę dla danej liczby elementów. Złożoność pamięciowa to optymalne $O(V + E)$.

Fragment kodu 3: Konstruktor i struktura klasy `AdjList`.

```

1  class AdjList : public Graph {
2  private:
3      // adj[v] = list of (neighbor, weight) pairs for vertex v
4      std::vector<std::vector<std::pair<int, int>>> adj;
5
6  public:
7      AdjList(int vertices) : Graph(vertices), adj(vertices) {}

```

```

8      // ...
9  };

```

Fragment kodu 4: Funkcja addEdge w reprezentacji listowej.

```

1  void AdjList::addEdge(int u, int v, int w) {
2      // Check if edge already exists and update it
3      for (auto& [neighbor, weight] : adj[u]) {
4          if (neighbor == v) {
5              weight = w;
6              return;
7          }
8      }
9      adj[u].emplace_back(v, w);
10     edgeCount++;
11 }

```

2.2 Macierz sąsiedztwa

W macierzy sąsiedztwa graf zapamiętywany jest z użyciem wektora dwuwymiarowego wielkości rzędu $V \times V$. Przy braku krawędzi, pod indeksem (u, v) znajduje się zdefiniowana z góry wartość symbolizująca nieskończoność (np. `INT_MAX`). Macierz sąsiedztwa cechuje duża szybkość dostępu do danej krawędzi ($O(1)$) kosztem zużytej pamięci w wysokości $O(V^2)$.

3 Testy efektywności

Automatyzacja pomiarów została uzyskana na bazie biblioteki `<chrono>` (w tym `std::chrono::high_resolution_clock`). Pomiary uśredniono ze 100 prób (wykonanych na generowanych losowo na gorąco instancjach) dla parametrów $V \in \{10, 50, 100, 500, 1000\}$ i gęstości rzędu 25%, 50%, 75%, 100%. Problem wyznaczenia ścieżek od wybranego wierzchołka źródłowego do wszystkich pozostałych zrealizowano korzystając z obu reprezentacji.

3.1 Wyniki pomiarów w formie tabelarycznej

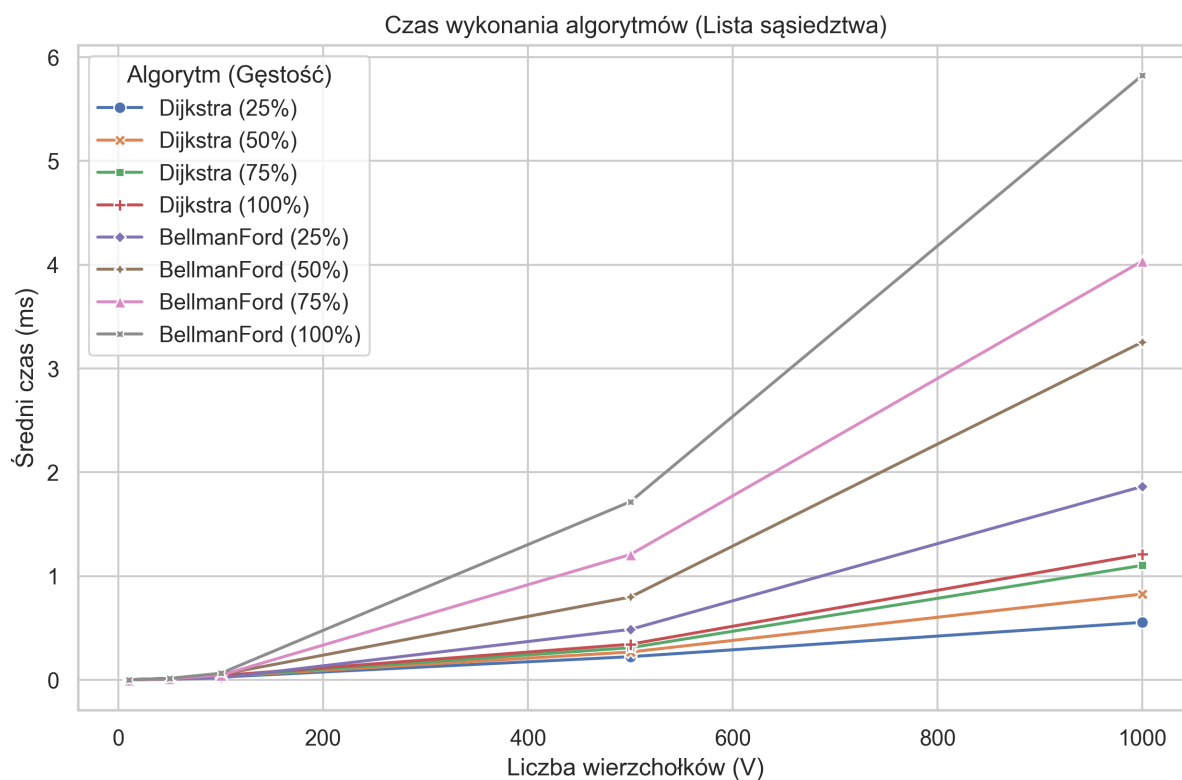
Zgodnie z wymaganiami projektowymi, szczegółowe wyniki zebrano i uśredniono w poniższej tabeli:

Tabela 1: Wyniki pomiarów czasu wykonania (w milisekundach)

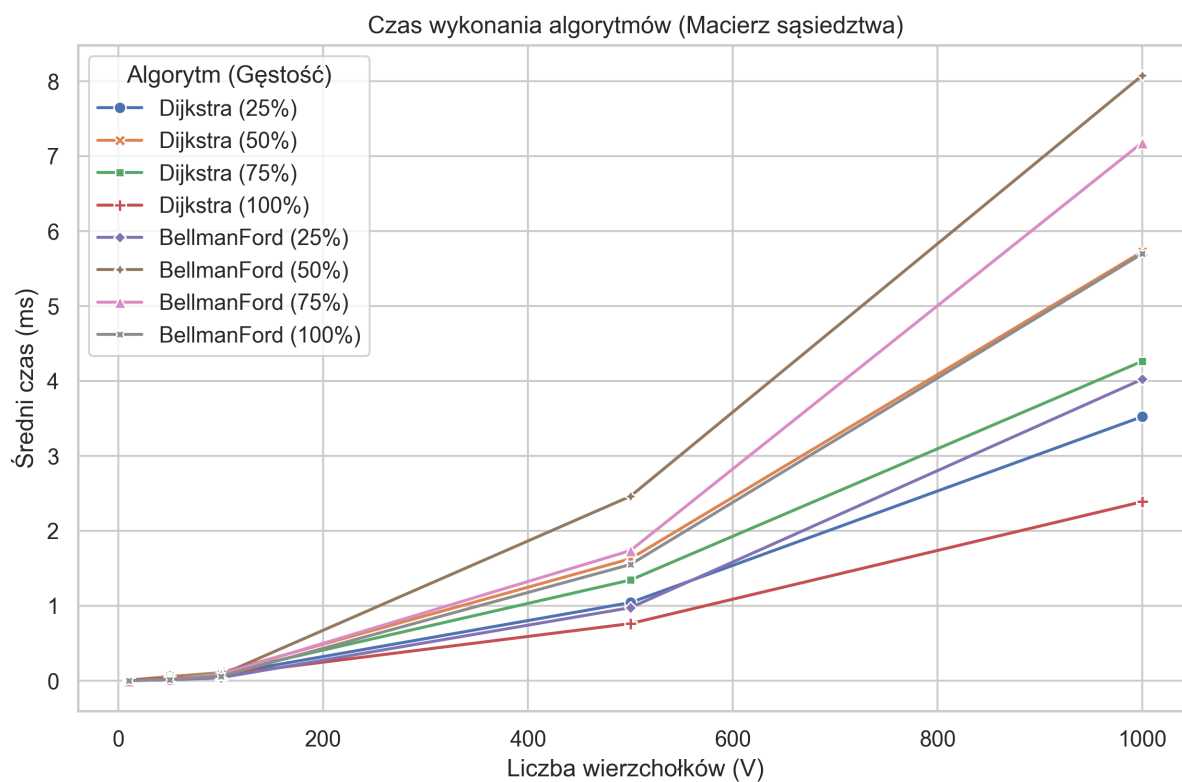
representation	algorithm	vertices density	10	50	100	500	1000
List	BellmanFord	25	0.00	0.00	0.02	0.49	1.86
		50	0.00	0.01	0.05	0.80	3.25
		75	0.00	0.01	0.04	1.21	4.04
		100	0.00	0.01	0.06	1.72	5.83
	Dijkstra	25	0.00	0.01	0.03	0.22	0.55
		50	0.00	0.01	0.03	0.27	0.83
		75	0.00	0.01	0.03	0.31	1.10
		100	0.00	0.01	0.04	0.34	1.21
Matrix	BellmanFord	25	0.00	0.01	0.04	0.97	4.02
		50	0.00	0.02	0.08	2.46	8.08
		75	0.00	0.02	0.09	1.74	7.18
		100	0.00	0.01	0.06	1.55	5.70
	Dijkstra	25	0.00	0.05	0.08	1.04	3.52
		50	0.01	0.06	0.11	1.63	5.73
		75	0.01	0.04	0.10	1.34	4.26
		100	0.01	0.03	0.08	0.76	2.39

3.2 Wykresy typu 1: Względem reprezentacji

Poniższe wykresy prezentują zależność czasu wykonania algorytmów (Dijkstry i Bellmana-Forda) na danej reprezentacji od rosnącej liczby punktów (wierzchołków) dla wszystkich gęstości (łącznie 8 linii na wykres).



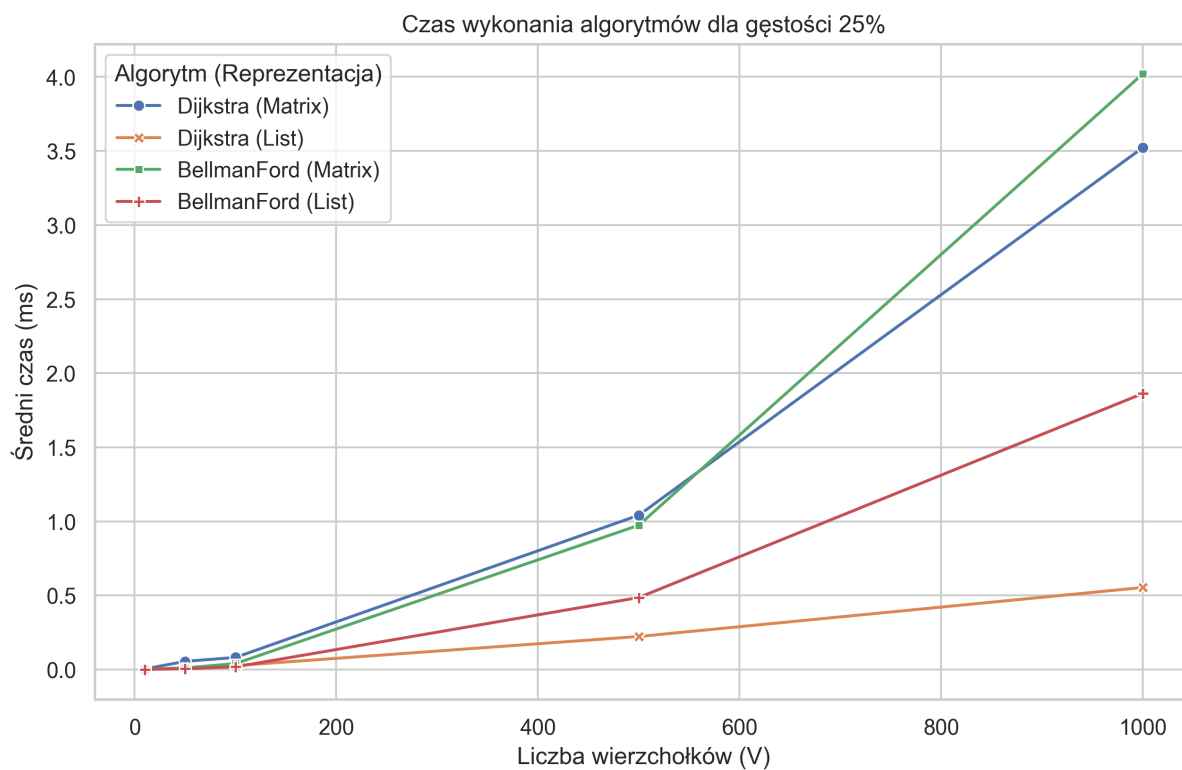
Rysunek 2: Czas wykonania w zależności od liczby wierzchołków (Lista Sąsiedztwa).



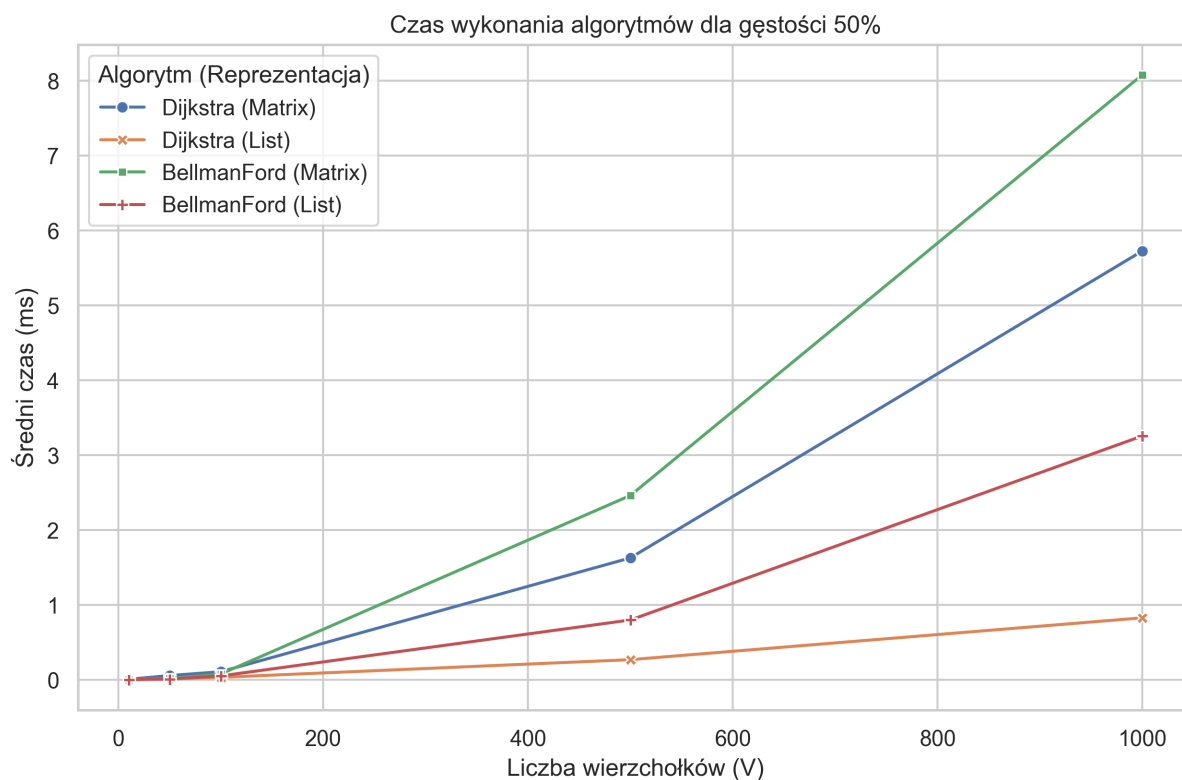
Rysunek 3: Czas wykonania w zależności od liczby wierzchołków (Macierz Sąsiedztwa).

3.3 Wykresy typu 2: Względem gęstości grafu

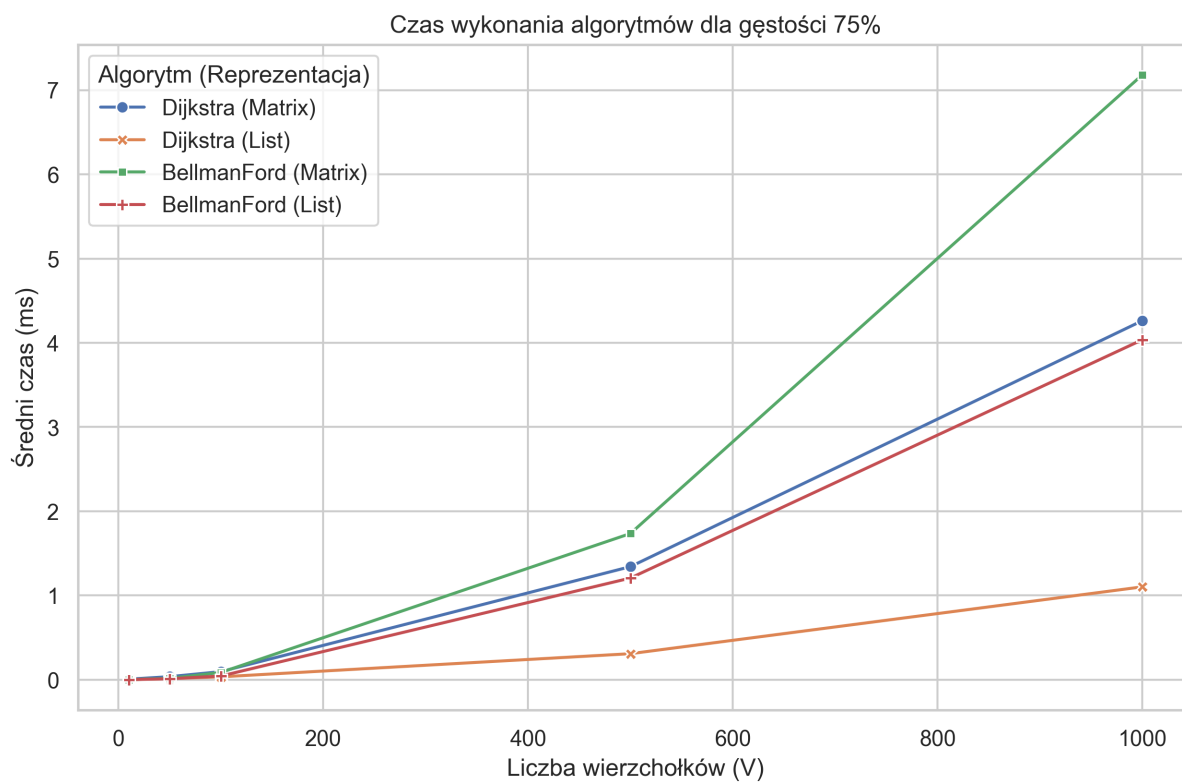
Wyniki dla poszczególnych gęstości ze szczegółowym wglądem w stopień wysycenia struktury krawędziami dla obu reprezentacji oraz algorytmów (łącznie 4 linie na wykres).



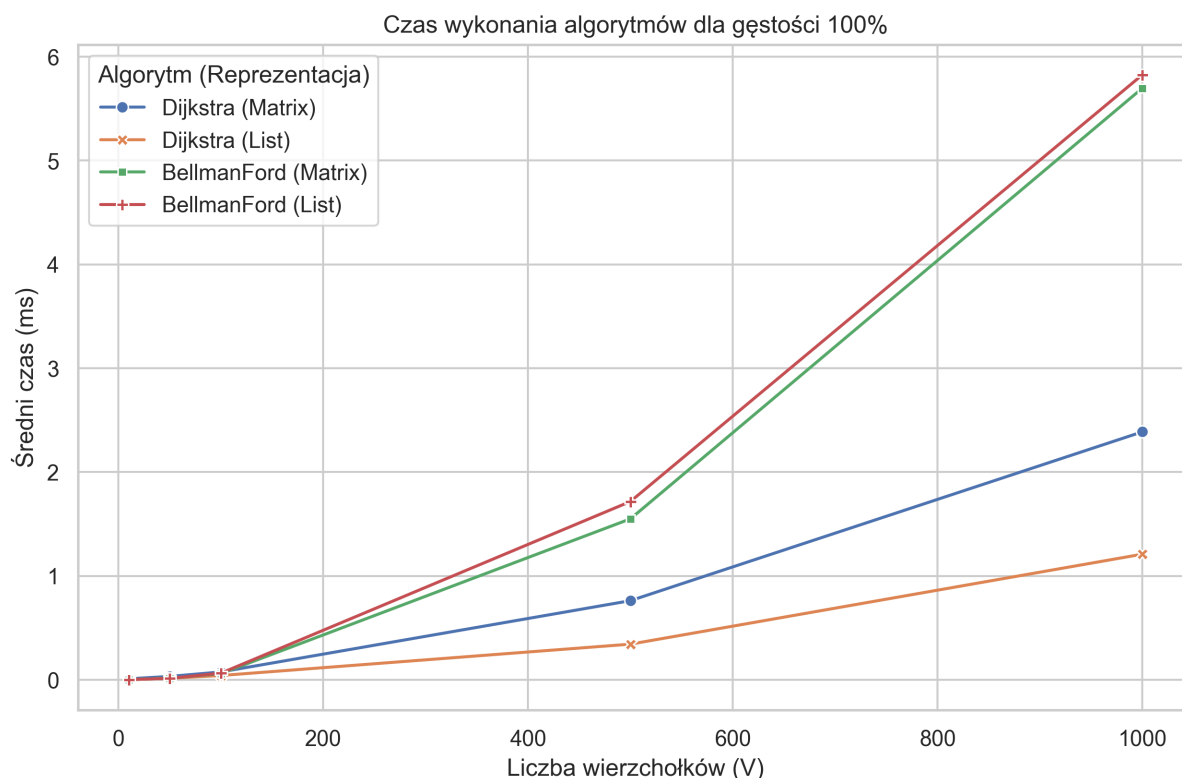
Rysunek 4: Czas wykonania w zależności od liczby wierzchołków dla gęstości 25%.



Rysunek 5: Czas wykonania w zależności od liczby wierzchołków dla gęstości 50%.



Rysunek 6: Czas wykonania w zależności od liczby wierzchołków dla gęstości 75%.



Rysunek 7: Czas wykonania w zależności od liczby wierzchołków dla gęstości 100%.

4 Wnioski

Interpretacja wyników poddanych analizie przynosi szereg spostrzeżeń:

- **Decydująca rola typu reprezentacji:** Wybór listy sąsiedztwa w znakomitej większości przypadków wpływa nader pozytywnie na ogół optymalizacji i obniżenie niezbędnego czasu działania (w szerszym ujęciu rzadszych połączeń między węzłami). Zastosowana do tego celu macierz sąsiedztwa nieodłącznie narzuca narzut operacyjny w postaci przymusowej, pełnej ewaluacji wiersza w macierzy – wysoce szkodliwy przy obszernych wymiarach tablicy w przypadku Dijkstry.
- **Czułość na parametr gęstości w Dijkstrze:** Zastosowany kopiec pozwala zbliżyć wydajność ogółu w kierunku zadowalającego rzędu logarytmicznego. Jednakże obfite uzupełnienie wagi skutkuje w praktyce rosnącą potrzebą sprawdzania wierzchołków i rozrastaniem się kolejki do priorytetyzacji (wydłużając czas trwania wprost proporcjonalnie do gęstości).
- **Rygor Bellmana-Forda:** Przeprowadzone pomiary ukazują powolność Bellmana-Forda. Przestarzały czasowo, relaksujący po wszystkich obecnych krawędziach mechanizm $O(V \cdot E)$ sprawia, że badany wskaźnik wykonania poszybował zauważalnie ujemnie wobec Dijkstry. Algorytm odczuwa rosnące powiązania w gęstym grafie na niesprzyjającym, ekstremalnym poziomie.
- **Podsumowanie:** Wyniki bezwzględnie dowodzą znacznej przewagi w zderzeniu wariantów. Kombinacja **algorytmu Dijkstry na odpowiednio przydzielonej liście sąsiedztwa stanowi filar i pożądany standard.**

5 Bibliografia

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Wprowadzenie do algorytmów*. PWN, Warszawa 2012.
2. Skrypt do przedmiotu Projektowanie i Analiza Algorytmów.
3. Bjarne Stroustrup. *Język C++*. WNT, Warszawa 2002.